

Chapter 1. Tabular MDP

Samsung AI Expert
Reinforcement Learning

Jul 7, 2025

Contents

What to do?

- Representation of Markov decision process
 - Basic matrix algebra using **numpy**
- Implementation of **value iteration**
- Implementation of **policy iteration**
- Practice : GridWorld

MDP - Review

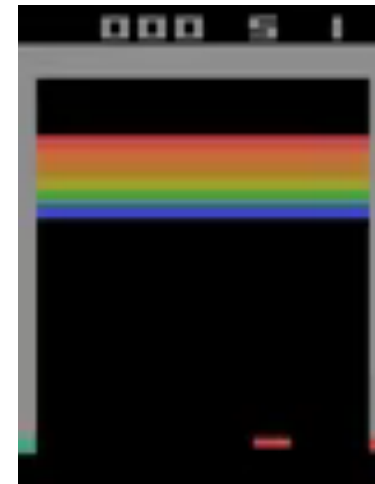
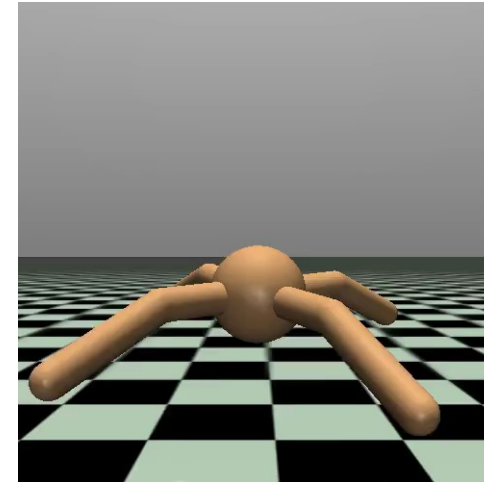
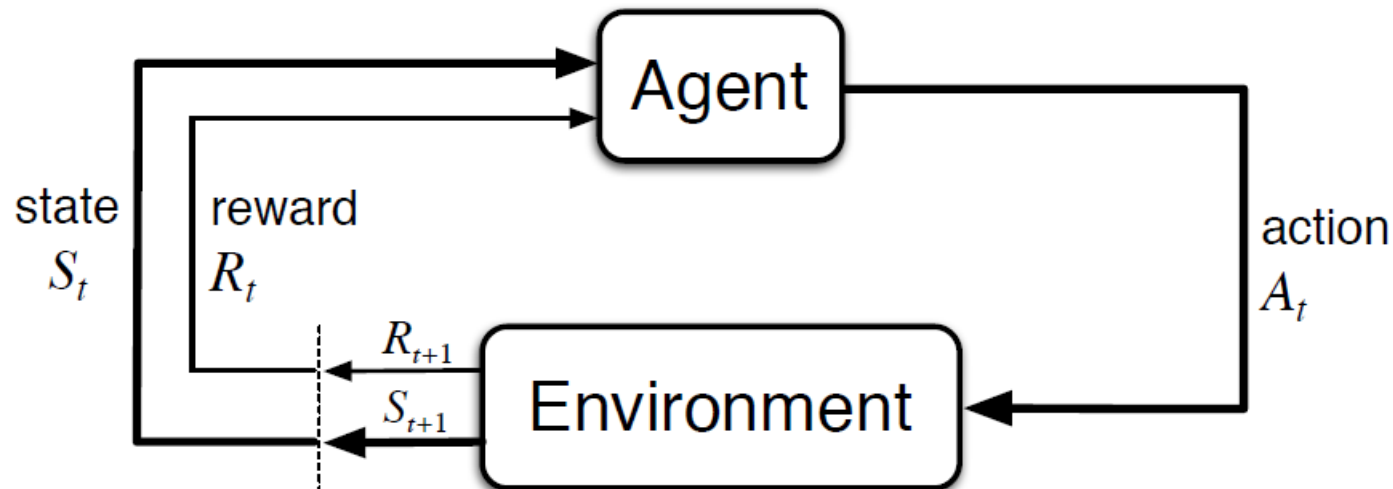
$\mathcal{S} = \{s_0, \dots, s_{n-1}\}$: state space

$\mathcal{A} = \{a_0, \dots, a_{m-1}\}$: action space

$p(s'|s, a)$: transition probability

$r(s, a)$: reward function

γ : discount rate



MDP - Review

How to represent these data?

1. transition probability $p(s'|s, a)$
2. reward function $r(s, a)$



⇒ We use **NumPy**, a Python package for linear algebra!

MDP - Review

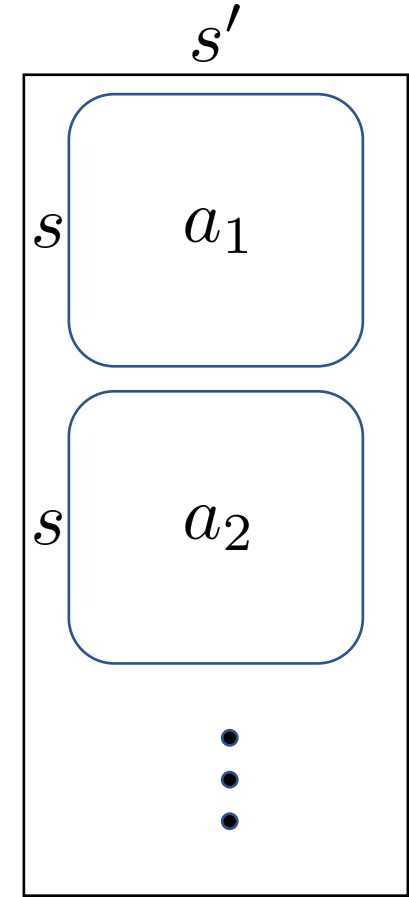
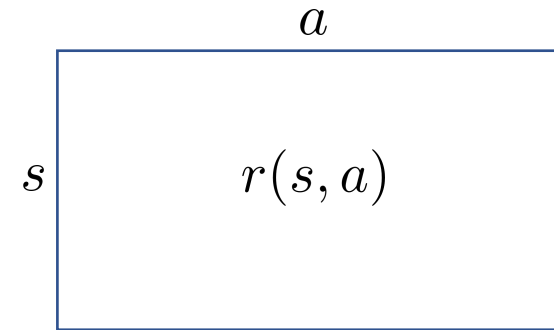
How to represent these data?

1. transition probability $p(s'|s, a)$

→ matrix P of size $|\mathcal{S}||\mathcal{A}| \times |\mathcal{S}|$

2. reward function $r(s, a)$

→ matrix R of size $|\mathcal{S}| \times |\mathcal{A}|$



MDP - Review

$$\mathcal{S} = \{s_0, s_1\}, \quad \mathcal{A} = \{a_0, a_1\},$$

$$r(s_0, a_0) = -2, \quad r(s_0, a_1) = -0.5,$$

$$r(s_1, a_0) = -1, \quad r(s_1, a_1) = -3.0,$$

$$p(s_0 | s_0, a_0) = 0.75,$$

$$p(s_0 | s_1, a_0) = 0.75,$$

$$p(s_0 | s_0, a_1) = 0.25,$$

$$p(s_0 | s_1, a_1) = 0.25.$$

```
1  R = np.array([[ -2.0, -0.5],
2                [ -1.0, -3.0]])
3
4  P = np.array([[ 0.75, 0.25],
5                [ 0.75, 0.25],
6                [ 0.25, 0.75],
7                [ 0.25, 0.75]])
```

Solving Tabular MDPs – Value Iteration

Review : **Bellman operator** $\mathcal{T} : \mathbb{R}^{\mathcal{S}} \rightarrow \mathbb{R}^{\mathcal{S}}$ is given by

$$(\mathcal{T}v)(s) = \max_a \left(r(s, a) + \gamma \sum_{s'} p(s'|s, a) v(s') \right)$$

Given a vector v of size $n \times 1$,

Step 1. compute $r(s, a) + \gamma \sum_{s'} p(s'|s, a) v(s')$,

Step 2. and then take $\max_a$.

TODO

- TODO 1
- TODO 2

TODO

- TODO 1

```
# TODO 1: Complete the following line.  
return R + gamma * np.reshape(P@v, newshape=R.shape, order='F')
```

- TODO 2

```
# TODO 2: Complete the following line.  
q = q_ftn(P, R, gamma, v)  
pi = np.argmax(q, axis=1)
```

Solving Tabular MDPs – Value Iteration

Step 1. compute $r(s, a) + \gamma \sum_{s'} p(s'|s, a)v(s')$,

```
1 def q_ftn(P, R, gamma, v):
2     """
3     given v, get corresponding q
4     """
5     return R + gamma * np.reshape(np.matmul(P, v), newshape=R.shape, order='F')
```

Shape of $q(s, a)$?

Step 2. and then take $\max_a$.

```
1 def bellman_update(P, R, gamma, v):
2     """
3     implementation of one-step Bellman update
4     return : vector of shape (|S|, 1) which corresponds to Tv, where T is Bellman operator
5     """
6     q = q_ftn(P, R, gamma, v)
7     v_next = np.max(q, axis=1, keepdims=True) # computation of Bellman operator Tv
8
9     return v_next
```

Solving Tabular MDPs – Value Iteration

$$\pi(s) = \arg \max_a \left(r(s, a) + \gamma \sum_{s'} p(s'|s, a) v(s') \right)$$

```
1 def greedy(P, R, gamma, v):
2     """
3     construct greedy policy by pi(s) = argmax_a q(s, a)
4     """
5     q = q_ftn(P, R, gamma, v)
6     pi = np.argmax(q, axis=1)
7
8     return pi
```

Combining all of these, we have...

Solving Tabular MDPs – Value Iteration

```
1  def VI(P, R, gamma):
2      """
3      implementation of value iteration
4      """
5      EPS = 1e-6
6      nS, nA = R.shape
7      # initialize v
8      v = np.zeros(shape=(nS, 1), dtype=np.float)
9
10     while True:
11         v_next = bellman_update(P, R, gamma, v)
12         if np.linalg.norm(v_next - v, ord=np.inf) < EPS:
13             break
14         v = v_next
15
16     pi = greedy(P, R, gamma, v)
17
18     return v, pi
```

(terminal condition)

$$\max_s |v(s) - (\mathcal{T}v)(s)| \leq \epsilon$$



TODO

- TODO 3

- TODO 4



TODO

- TODO 3

```
# TODO 3: Fill the blank below.  
if np.linalg.norm(v_next - v, ord=np.inf) < EPS:  
    break  
v = v_next
```

- TODO 4

```
# TODO 4: Construct policy obtained  $v^*$  .  
pi = greedy(P, R, gamma, v)
```

Solving Tabular MDPs – Value Iteration

```
1  def VI(P, R, gamma):
2      """
3      implementation of value iteration
4      """
5      EPS = 1e-6
6      nS, nA = R.shape
7      # initialize v
8      v = np.zeros(shape=(nS, 1), dtype=np.float)
9
10     while True:
11         v_next = bellman_update(P, R, gamma, v)
12         if np.linalg.norm(v_next - v, ord=np.inf) < EPS:
13             break
14         v = v_next
15
16     pi = greedy(P, R, gamma, v)
17
18     return v, pi
```

(terminal condition)

$$\max_s |v(s) - (\mathcal{T}v)(s)| \leq \epsilon$$



Solving Tabular MDPs – Policy Iteration

Review : any policy π satisfies

$$v^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) v^\pi(s').$$

Step 1. compute v^π by solving the above equation (Policy Evaluation)

Step 2. determine π_{next} greedily (Policy Improvement):

$$\pi_{\text{next}}(s) = \arg \max_a \left(r(s, a) + \gamma \sum_{s'} p(s'|s, a) v^\pi(s') \right)$$

Solving Tabular MDPs – Policy Iteration

Step 1. compute v^π by solving the above equation (Policy Evaluation)

```
1 def induced_dynamic(nS, P, R, pi):
2     """
3     given policy pi, compute induced dynamic P^pi & R^pi
4     """
5     S = range(nS)
6     rows = np.arange(nS) + nS * pi
7     P_pi = P[rows]
8     R_pi = np.array([[R[s, pi[s]]] for s in range(nS)])
9
10    return P_pi, R_pi
```

```
1 def eval_policy(nS, P, R, gamma, pi):
2     """
3     policy evaluation
4     """
5     P_pi, R_pi = induced_dynamic(nS, P, R, pi)
6
7     Id = np.identity(nS)
8
9     # discounted reward problem
10    v_pi = np.linalg.solve(Id - gamma * P_pi, R_pi)
11    return v_pi
```

$$P^\pi = \begin{pmatrix} p(0|0, \pi(0)) & \cdots & p(n-1|0, \pi(0)) \\ \vdots & \vdots & \vdots \\ p(0|n-1, \pi(n-1)) & \cdots & p(n-1|n-1, \pi(n-1)) \end{pmatrix}$$

$$r^\pi = \begin{pmatrix} r(0, \pi(0)) \\ \vdots \\ r(n-1, \pi(n-1)) \end{pmatrix}$$

$$v^\pi = r^\pi + \gamma P^\pi v^\pi$$

$\downarrow$

$$(I - \gamma P^\pi) v^\pi = r^\pi$$



TODO

- TODO 5

- TODO 6



TODO

- TODO 5

```
# TODO 5: Compute 'row number' by formulations above.  
rows = np.arange(nS) + nS * pi
```

- TODO 6

```
# TODO 6: Get P_pi and R_pi from P and R.  
P_pi = P[rows]  
R_pi = np.array([[R[s, pi[s]]] for s in range(nS)])
```

Solving Tabular MDPs – Policy Iteration

Step 1. compute v^π by solving the above equation (Policy Evaluation)

```
1 def induced_dynamic(nS, P, R, pi):
2     """
3     given policy pi, compute induced dynamic P^pi & R^pi
4     """
5     S = range(nS)
6     rows = np.arange(nS) + nS * pi
7     P_pi = P[rows]
8     R_pi = np.array([[R[s, pi[s]]] for s in range(nS)])
9
10    return P_pi, R_pi
```

```
1 def eval_policy(nS, P, R, gamma, pi):
2     """
3     policy evaluation
4     """
5     P_pi, R_pi = induced_dynamic(nS, P, R, pi)
6
7     Id = np.identity(nS)
8
9     # discounted reward problem
10    v_pi = np.linalg.solve(Id - gamma * P_pi, R_pi)
11    return v_pi
```

$$P^\pi = \begin{pmatrix} p(0|0, \pi(0)) & \cdots & p(n-1|0, \pi(0)) \\ \vdots & \vdots & \vdots \\ p(0|n-1, \pi(n-1)) & \cdots & p(n-1|n-1, \pi(n-1)) \end{pmatrix}$$

$$r^\pi = \begin{pmatrix} r(0, \pi(0)) \\ \vdots \\ r(n-1, \pi(n-1)) \end{pmatrix}$$

$$v^\pi = r^\pi + \gamma P^\pi v^\pi$$

$\downarrow$

$$(I - \gamma P^\pi) v^\pi = r^\pi$$



TODO

- TODO 7

TODO

- TODO 7

```
# TODO 7: update policy
pi_next = greedy(P, R, gamma, v)
if (pi_next == pi).all():
    break
pi = pi_next
```

Solving Tabular MDPs – Policy Iteration

```
1  def PI(P, R, gamma):
2      """
3      implementation of policy iteration
4      """
5      nS, nA = R.shape
6
7      # initialize policy
8      pi = np.random.randint(nA, size=nS)
9
10     while True:
11         v = eval_policy(nS, P, R, gamma, pi)
12         pi_next = greedy(P, R, gamma, v)
13         if (pi_next == pi).all():
14             break
15         pi = pi_next
16
17     return v, pi
```

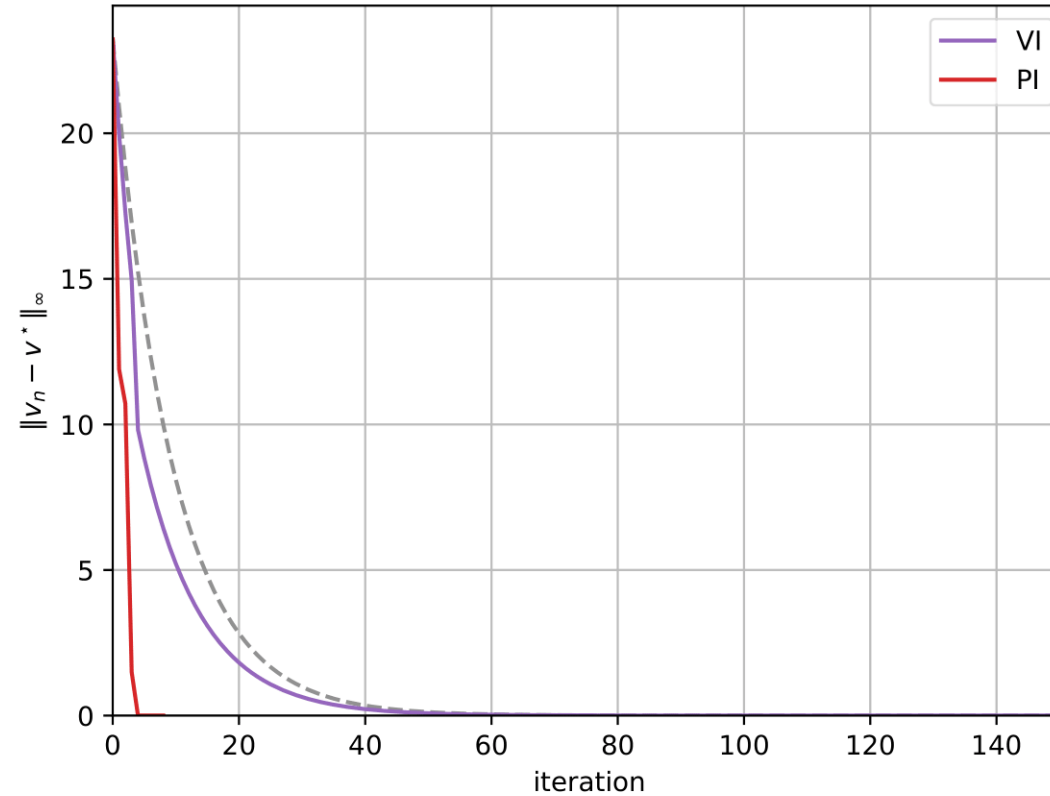
terminal condition : $\pi_{k+1} = \pi_k$

Example : GridWorld

					(1, 6) +10	
(4, 1)						
				(7, 5)		

- 4 actions available : **UP**, **LEFT**, **RIGHT**, **DOWN**
- If **LEFT** is chosen at the leftmost cell, get penalty -1 and stay.
- If you reach (1,6), any action done in the cell takes you to the cell (4,1) with reward 10.
- (7,5) : another interesting cell in this world; teleports you to (1,6) regardless of the action you choose.

Example : GridWorld



Thank you!



Learning and Decision Systems